

DDA3020 Homework 1 Solution

1 Written Problems (50 points)

1.1. (Weighted Linear Regression, 10 points)

(a) Closed-form Solution:

Let's augment the input vector to absorb the bias. Define $\tilde{\mathbf{x}}_i = [\mathbf{x}_i^T, 1]^T \in \mathbb{R}^{d+1}$ and the augmented weight matrix $\tilde{\mathbf{W}} = [\mathbf{W}, \mathbf{b}]$. Then the prediction model becomes $\tilde{\mathbf{W}}\tilde{\mathbf{x}}_i = \mathbf{W}\mathbf{x}_i + \mathbf{b}$.

The objective function can be rewritten as (1 points):

$$J(\tilde{\mathbf{W}}) = \sum_{i=1}^N \alpha_i \|\mathbf{y}_i - \tilde{\mathbf{W}}\tilde{\mathbf{x}}_i\|^2$$

Let $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ be the design matrix where the i -th row is $\tilde{\mathbf{x}}_i^T$. Let $\mathbf{Y} \in \mathbb{R}^{N \times m}$ be the target matrix where the i -th row is $\mathbf{y}_i^T$. Let $\mathbf{A} = \text{diag}(\alpha_1, \dots, \alpha_N)$ be a diagonal matrix of weights.

The loss function in matrix notation is (1 points):

$$J(\tilde{\mathbf{W}}) = \text{Tr} \left((\mathbf{Y} - \mathbf{X}\tilde{\mathbf{W}}^T)^T \mathbf{A} (\mathbf{Y} - \mathbf{X}\tilde{\mathbf{W}}^T) \right)$$

To find the minimum, we take the derivative with respect to $\tilde{\mathbf{W}}$ and set it to zero:

$$\begin{aligned} \nabla_{\tilde{\mathbf{W}}} J(\tilde{\mathbf{W}}) &= -2(\mathbf{Y} - \mathbf{X}\tilde{\mathbf{W}}^T)^T \mathbf{A} \mathbf{X} = 0 \\ (\mathbf{Y}^T - \tilde{\mathbf{W}}\mathbf{X}^T) \mathbf{A} \mathbf{X} &= 0 \\ \mathbf{Y}^T \mathbf{A} \mathbf{X} &= \tilde{\mathbf{W}}\mathbf{X}^T \mathbf{A} \mathbf{X} \\ \tilde{\mathbf{W}} &= \mathbf{Y}^T \mathbf{A} \mathbf{X} (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1} \end{aligned}$$

Thus, the optimal solution is (3 points):

$$\boxed{\tilde{\mathbf{W}} = \mathbf{Y}^T \mathbf{A} \mathbf{X} (\mathbf{X}^T \mathbf{A} \mathbf{X})^{-1}}$$

(b) Gradient Descent:

We compute the gradients with respect to $\mathbf{W}$ and $\mathbf{b}$ separately from the original objective function:

$$J(\mathbf{W}, \mathbf{b}) = \sum_{i=1}^N \alpha_i \|\mathbf{y}_i - (\mathbf{W}\mathbf{x}_i + \mathbf{b})\|^2$$

Gradients (4 points, 2 points each):

$$\frac{\partial J}{\partial \mathbf{W}} = \sum_{i=1}^N \alpha_i \cdot 2(\mathbf{W}\mathbf{x}_i + \mathbf{b} - \mathbf{y}_i) \mathbf{x}_i^T$$

$$\frac{\partial J}{\partial \mathbf{b}} = \sum_{i=1}^N \alpha_i \cdot 2(\mathbf{W}\mathbf{x}_i + \mathbf{b} - \mathbf{y}_i)$$

Update Rules (with learning rate η) (1 points, 0.5 points each):

$$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} - 2\eta \sum_{i=1}^N \alpha_i (\mathbf{W}^{(t)} \mathbf{x}_i + \mathbf{b}^{(t)} - \mathbf{y}_i) \mathbf{x}_i^T$$

$$\mathbf{b}^{(t+1)} \leftarrow \mathbf{b}^{(t)} - 2\eta \sum_{i=1}^N \alpha_i (\mathbf{W}^{(t)} \mathbf{x}_i + \mathbf{b}^{(t)} - \mathbf{y}_i)$$

1.2. (Mathematical Foundations: Optimization & Information Theory, 15 points)**(a) Convexity, Lagrangian, and KKT Conditions:**

Convexity: (2 points) The Hessian matrix is $\nabla^2 f_0 = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$, which is positive definite ($\succ 0$).

Thus, f_0 is strictly convex. The constraints are linear inequalities ($x_1 + x_2 - 1 \leq 0$, $-x_1 \leq 0$, $-x_2 \leq 0$), which define a convex set (a polygon). Since we are minimizing a convex function over a convex set, this is a **convex optimization problem**.

Lagrangian: (1 points) Let the constraints be $g_1(\mathbf{x}) = x_1 + x_2 - 1 \leq 0$, $g_2(\mathbf{x}) = -x_1 \leq 0$, and $g_3(\mathbf{x}) = -x_2 \leq 0$. The Lagrangian $L(\mathbf{x}, \mathbf{u})$ with multipliers $\mathbf{u} = [u_1, u_2, u_3]^T \in \mathbb{R}^3$ is:

$$L(x_1, x_2, u_1, u_2, u_3) = (x_1 - 2)^2 + (x_2 - 1)^2 + u_1(x_1 + x_2 - 1) - u_2x_1 - u_3x_2$$

KKT Conditions: (2 points)

1. **Stationarity:** $\nabla_{\mathbf{x}} L = 0$

$$\frac{\partial L}{\partial x_1} = 2(x_1 - 2) + u_1 - u_2 = 0 \implies 2(x_1 - 2) = u_2 - u_1$$

$$\frac{\partial L}{\partial x_2} = 2(x_2 - 1) + u_1 - u_3 = 0 \implies 2(x_2 - 1) = u_3 - u_1$$

2. Primal Feasibility:

$$x_1 + x_2 \leq 1, \quad x_1 \geq 0, \quad x_2 \geq 0$$

3. Dual Feasibility:

$$u_1 \geq 0, \quad u_2 \geq 0, \quad u_3 \geq 0$$

4. Complementary Slackness:

$$u_1(x_1 + x_2 - 1) = 0, \quad u_2x_1 = 0, \quad u_3x_2 = 0$$

(b) Solving for Optimal Point $\mathbf{x}^*$:

We analyze the geometry. The unconstrained minimum is at $(2, 1)$, which violates $x_1 + x_2 \leq 1$. Thus, the solution lies on the boundary $x_1 + x_2 = 1$. From Stationarity, we have $2(x_1 - 2) = u_2 - u_1$ and $2(x_2 - 1) = u_3 - u_1$. Assuming only the first constraint is active ($u_1 > 0, u_2 = 0, u_3 = 0$), we simplify to:

$$2(x_1 - 2) = -u_1 \quad \text{and} \quad 2(x_2 - 1) = -u_1$$

Equating them gives $x_1 - 2 = x_2 - 1 \implies x_1 - x_2 = 1$. Combining with the boundary $x_1 + x_2 = 1$, we get $x_1 = 1, x_2 = 0$ (3 points). Checking KKT conditions at $\mathbf{x}^* = (1, 0)$:

- Feasibility: $1 + 0 \leq 1, 1 \geq 0, 0 \geq 0$. OK.
- Multipliers: From stationarity, $u_1 = -2(1 - 2) = 2 \geq 0$. With $x_2 = 0$ active, we can allow $u_3 \geq 0$. Here $u_3 = 0$ works (since $2(0 - 1) + 2 - u_3 = 0 \implies u_3 = 0$). OK.

Thus, the optimal solution is (2 points):

$$\boxed{\mathbf{x}^* = (1, 0)}$$

(c) KL Divergence Properties

(1) **Prove Asymmetry:** (1 points) The definition of KL Divergence for discrete distributions P and Q is:

$$D_{KL}(P||Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

$$D_{KL}(Q||P) = \sum_x Q(x) \log \frac{Q(x)}{P(x)}$$

In general, $\sum P(x) \log P(x) - \sum P(x) \log Q(x) \neq \sum Q(x) \log Q(x) - \sum Q(x) \log P(x)$. Therefore, $D_{KL}(P||Q) \neq D_{KL}(Q||P)$.

(2) **Counter-example:** Let $X = \{0, 1\}$. Let $P(0) = 0.5, P(1) = 0.5$. Let $Q(0) = 0.9, Q(1) = 0.1$.

Calculate $D_{KL}(P||Q)$ (2 points):

$$\begin{aligned} D_{KL}(P||Q) &= 0.5 \ln \frac{0.5}{0.9} + 0.5 \ln \frac{0.5}{0.1} \\ &= 0.5 \ln \frac{5}{9} + 0.5 \ln 5 \\ &\approx 0.5(-0.5878) + 0.5(1.6094) \approx -0.2939 + 0.8047 = 0.5108 \end{aligned}$$

Calculate $D_{KL}(Q||P)$ (2 points):

$$\begin{aligned} D_{KL}(Q||P) &= 0.9 \ln \frac{0.9}{0.5} + 0.1 \ln \frac{0.1}{0.5} \\ &= 0.9 \ln 1.8 + 0.1 \ln 0.2 \\ &\approx 0.9(0.5878) + 0.1(-1.6094) \approx 0.5290 - 0.1609 = 0.3681 \end{aligned}$$

Since $0.5108 \neq 0.3681$, we have shown $D_{KL}(P||Q) \neq D_{KL}(Q||P)$.

1.3. (Logistic Regression, 25 points)

(a) Derivative of the Sigmoid Function

We are given $\sigma(z) = (1 + e^{-z})^{-1}$. Applying the chain rule (3 points):

$$\begin{aligned} \sigma'(z) &= \frac{d}{dz}(1 + e^{-z})^{-1} \\ &= -1 \cdot (1 + e^{-z})^{-2} \cdot \frac{d}{dz}(1 + e^{-z}) \\ &= -(1 + e^{-z})^{-2} \cdot (-e^{-z}) \\ &= \frac{e^{-z}}{(1 + e^{-z})^2} \end{aligned}$$

Rearranging the result algebraically (2 points):

$$\frac{e^{-z}}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \left(\frac{(1 + e^{-z}) - 1}{1 + e^{-z}} \right) = \sigma(z)(1 - \sigma(z))$$

Conclusion: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$.

(b) Gradient of the Cost Function

Let $z_i = \mathbf{w}^T \mathbf{x}_i$. By the chain rule (1 points):

$$\frac{\partial J}{\partial \mathbf{w}} = \sum_{i=1}^n \frac{\partial J_i}{\partial h_i} \cdot \frac{\partial h_i}{\partial z_i} \cdot \frac{\partial z_i}{\partial \mathbf{w}}$$

Partial of Cost w.r.t Prediction: (1 points)

$$\frac{\partial J_i}{\partial h_i} = - \left(\frac{y_i}{h_i} - \frac{1 - y_i}{1 - h_i} \right) = \frac{h_i - y_i}{h_i(1 - h_i)}$$

Partial of Prediction w.r.t z : (1 points) From Part (a), $\frac{\partial h_i}{\partial z_i} = h_i(1 - h_i)$.

Partial of z w.r.t $\mathbf{w}$: (1 points) $\frac{\partial z_i}{\partial \mathbf{w}} = \mathbf{x}_i$.

Combining these terms, the denominators cancel out (1 points):

$$\frac{\partial J_i}{\partial \mathbf{w}} = \frac{h_i - y_i}{h_i(1 - h_i)} \cdot h_i(1 - h_i) \cdot \mathbf{x}_i = (h_i - y_i)\mathbf{x}_i$$

Summing over all samples and converting to matrix notation:

$$\nabla_{\mathbf{w}} J(\mathbf{w}) = \mathbf{X}^T(\mathbf{h} - \mathbf{y})$$

(c) Hessian and Convexity (2 points)

The Hessian is the derivative of the gradient. Since $\mathbf{y}$ is constant:

$$\mathbf{H} = \nabla_{\mathbf{w}}(\mathbf{X}^T \mathbf{h}) = \mathbf{X}^T(\nabla_{\mathbf{w}} \mathbf{h})$$

The term $\nabla_{\mathbf{w}} \mathbf{h}$ represents the Jacobian of the prediction vector. Since h_i depends on $\mathbf{w}^T \mathbf{x}_i$:

$$\frac{\partial h_i}{\partial \mathbf{w}} = h_i(1 - h_i)\mathbf{x}_i^T$$

Thus, $\nabla_{\mathbf{w}} \mathbf{h} = \mathbf{S}\mathbf{X}$, where $\mathbf{S} \in \mathbb{R}^{n \times n}$ is a diagonal matrix with entries $S_{ii} = h_i(1 - h_i)$.

$$\mathbf{H} = \mathbf{X}^T \mathbf{S} \mathbf{X}$$

Proof of Convexity (PSD): (3 points)

For any non-zero vector $\mathbf{v} \in \mathbb{R}^{d+1}$:

$$\mathbf{v}^T \mathbf{H} \mathbf{v} = \mathbf{v}^T \mathbf{X}^T \mathbf{S} \mathbf{X} \mathbf{v} = (\mathbf{X} \mathbf{v})^T \mathbf{S} (\mathbf{X} \mathbf{v}) = \sum_{i=1}^n S_{ii} (\mathbf{x}_i^T \mathbf{v})^2$$

Since the sigmoid output $h_i \in (0, 1)$, we have $S_{ii} = h_i(1 - h_i) > 0$. The squared term $(\mathbf{x}_i^T \mathbf{v})^2 \geq 0$. Therefore, $\mathbf{v}^T \mathbf{H} \mathbf{v} \geq 0$, which implies $\mathbf{H}$ is Positive Semi-Definite and $J(\mathbf{w})$ is convex.

(d) Regularized Update Rule

The gradient of the regularization term $\frac{\lambda}{2} \mathbf{w}^T \mathbf{w}$ is $\lambda \mathbf{w}$ (2 points).

$$\nabla J_{reg}(\mathbf{w}) = \mathbf{X}^T(\mathbf{h} - \mathbf{y}) + \lambda \mathbf{w}$$

The Gradient Descent update rule is $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \nabla J_{reg}(\mathbf{w}^{(t)})$. Substituting the gradient (2 points):

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \alpha \left[\mathbf{X}^T(\mathbf{h}^{(t)} - \mathbf{y}) + \lambda \mathbf{w}^{(t)} \right]$$

Rearranging to show “weight decay” (1 points):

$$\mathbf{w}^{(t+1)} = (1 - \alpha\lambda) \mathbf{w}^{(t)} - \alpha \mathbf{X}^T(\mathbf{h}^{(t)} - \mathbf{y})$$

(e) Softmax Regression Gradient

We differentiate the loss $L_i = -\sum_{j=1}^K y_{ij} \log P_{ij}$ with respect to $\mathbf{w}_k$. Note that $P_{ij} = \frac{e^{z_{ij}}}{\sum_l e^{z_{il}}}$, so the denominator depends on $\mathbf{w}_k$ for all classes j .

Using the derivative of the logarithm: $\frac{\partial \log P_{ij}}{\partial \mathbf{w}_k} = \frac{1}{P_{ij}} \frac{\partial P_{ij}}{\partial \mathbf{w}_k}$.

Standard derivative of Softmax probability P_{ij} w.r.t logit z_{ik} (2 points):

$$\frac{\partial P_{ij}}{\partial z_{ik}} = P_{ij}(\delta_{jk} - P_{ik}) \quad \text{where } \delta_{jk} \text{ is Kronecker delta.}$$

Thus by chain rule (where $\frac{\partial z_{ik}}{\partial \mathbf{w}_k} = \mathbf{x}_i$):

$$\frac{\partial \log P_{ij}}{\partial \mathbf{w}_k} = (\delta_{jk} - P_{ik}) \mathbf{x}_i$$

Substituting back into the loss sum (2 points):

$$\begin{aligned} \nabla_{\mathbf{w}_k} L_i &= -\sum_{j=1}^K y_{ij} (\delta_{jk} - P_{ik}) \mathbf{x}_i \\ &= -\left[\left(\sum_{j=1}^K y_{ij} \delta_{jk} \right) - \left(\sum_{j=1}^K y_{ij} \right) P_{ik} \right] \mathbf{x}_i \end{aligned}$$

Using the properties of one-hot vectors ($\sum_j y_{ij} \delta_{jk} = y_{ik}$ and $\sum_j y_{ij} = 1$):

$$\nabla_{\mathbf{w}_k} L_i = -[y_{ik} - P_{ik}] \mathbf{x}_i = (P_{ik} - y_{ik}) \mathbf{x}_i$$

Summing over all samples (1 points):

$$\nabla_{\mathbf{w}_k} J(\mathbf{W}) = \sum_{i=1}^n (P(y_i = k | \mathbf{x}_i) - \mathbb{I}(y_i = k)) \mathbf{x}_i$$

2 Programming (50 points)

2.1. (Gradient Descent from Scratch, 25 points) Only allow using numpy for this question. Using torch or other deep learning frameworks will get zero points.

- (a) **(15 points)** Refer to `hw1_q2_1.py` for the code template. Implement correct gradient function (5 points), implement correct gradient descent update rule and for loop (5 points), Obtain the correct result (0, 0) (5 points)
- (b) **(5 points)**

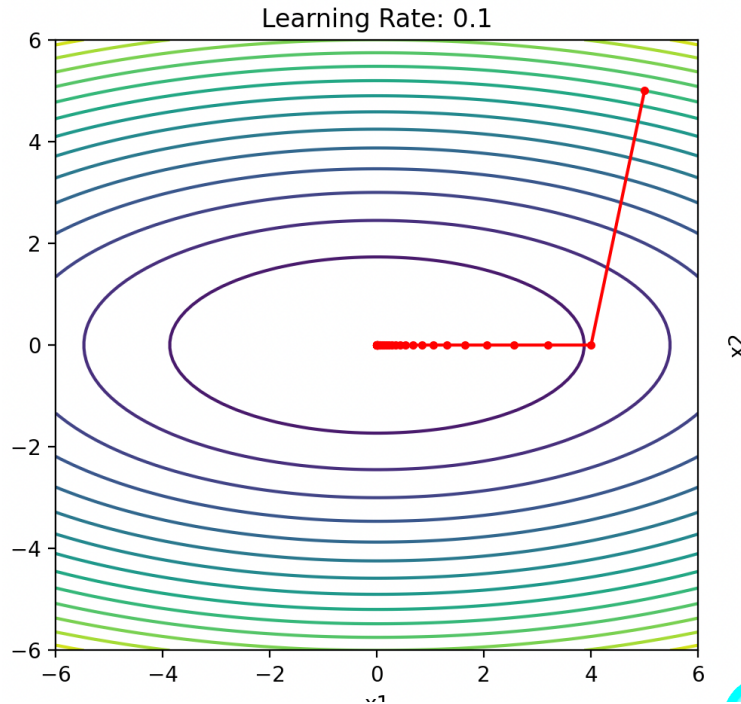


Figure 1: Gradient Descent Trajectory with Learning Rate $\eta = 0.1$

- (c) **(5 points)** Given convergence criterion (norm of gradient less than $1e-5$), 0.01 converges at 685, 0.1 converges at 63 and 1.1 diverges (**3 points**). The large learning rate (1.1) causes divergence (**2 points**).

2.2. (Logistic Regression, 25 points) 1. Only allow using numpy for this question. Using torch or other deep learning frameworks will get zero points. 2. The results may vary slightly due to random seeding for dataset splitting and initialization. As long as the train/test accuracy are around the expected values and the results obtained from code are consistent with the results reported in the PDF file, full points will be awarded.

- (a) **(10 points)** Refer to `hw1_q2_2.py` for the code template. Implement correct dataset splitting (**2 points**), implement correct data preprocessing (**2 points**), implement correct gradient and objective function (**2 points**), Implement correct gradient descent update rule and for loop (**2 points**), Obtain the correct result (train/test accuracy: around 0.975/0.967) (**2 points**)

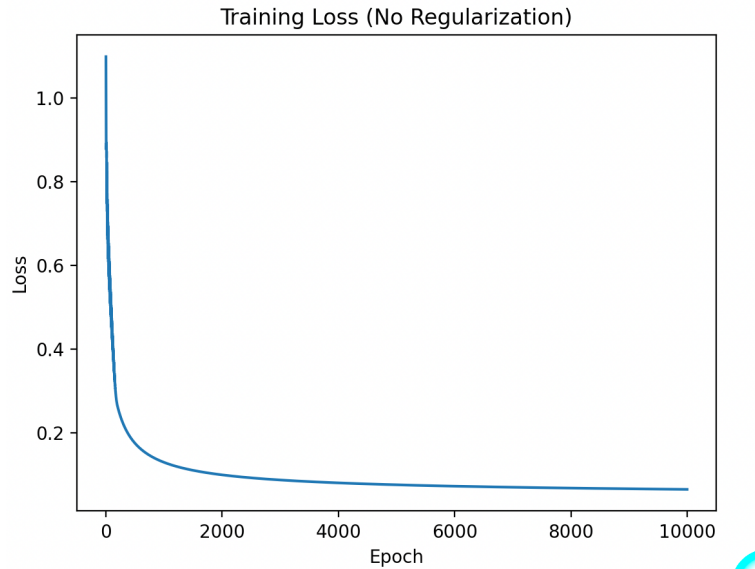


Figure 2: Gradient Descent Trajectory with Learning Rate $\eta = 0.1$

(b) **(5 points)** Correctly implement the regularized loss, compute its gradient, and apply the update rule **(4 points)**. Report the resulting train/test accuracy for different regularization strengths **(1 points)**:

- For $\lambda = 0.01$: **Train accuracy** ≈ 0.983 , **Test accuracy** ≈ 1.00
- For $\lambda = 0.1$: **Train accuracy** ≈ 0.908 , **Test accuracy** ≈ 0.90
- For $\lambda = 1.0$: **Train accuracy** ≈ 0.67 , **Test accuracy** ≈ 0.67

(c) **(5 points)** Correctly implement the regularized loss, compute its gradient, and apply the update rule **(4 points)**. Report the resulting train/test accuracy for different regularization strengths **(1 points)**:

- For $\lambda = 0.01$: **Train accuracy** ≈ 0.983 , **Test accuracy** ≈ 1.00
- For $\lambda = 0.1$: **Train accuracy** ≈ 0.775 , **Test accuracy** ≈ 0.767
- For $\lambda = 1.0$: **Train accuracy** ≈ 0.333 , **Test accuracy** ≈ 0.333

(d) **(5 points)** Comparison of L1 and L2 Regularization:

Model Performance: (2 points)

L2 regularization (Ridge) generally yields higher train and test accuracy than L1 (Lasso) for the same λ values, as seen in the results above. For example, with $\lambda = 0.1$, L2 achieves test accuracy ≈ 0.90 , while L1 drops to ≈ 0.77 . With large λ , both methods underfit, but L1 degrades more rapidly.

Sparsity of Weights: (3 points)

L1 regularization encourages sparsity in the learned weights, often driving many coefficients exactly to zero. This leads to simpler models that may ignore less informative features. In contrast, L2 regularization shrinks weights smoothly but rarely makes them exactly zero, so all features typically retain some influence.